

Norges teknisk-naturvitenskapelige
universitet
Institutt for datateknologi og
informatikk

TDT4102 Prosedyre-
og objektorientert
programmering
Vår 2026

Øving 7

Frist: 2026-03-01

Mål for denne øvingen:

- Introduksjon til pekere
- Dynamisk minnehåndtering med pekere
- Implementering av kopikonstruktør og `operator=`

Generelle krav:

- Bruk de eksakte navn og spesifikasjoner gitt i oppgaven.
- Denne øvingen skal implementeres uten hjelp fra `std_lib_facilities.h`.
- 70% av øvingen må godkjennes for at den skal vurderes som bestått.
- Øvingen skal leveres på [INGInious](#).
- Benytt VS Code til å skrive, kompilere og kjøre kode.

Anbefalt lesestoff:

- Kapittel 16 og 17 i PPP.

I denne øvingen skal du **ikke** benytte `std_lib_facilities.h`. Du må derfor inkludere bibliotekene du trenger, og holde orden på navnerom selv.

Nyttig å vite: Navnerom

En navnekonflikt oppstår hvis flere forskjellige ting har samme navn. Når et navn refererer til flere ting vet ikke kompilatoren hva den skal gjøre. Navnerom (`namespace`) brukes for forhindre slike navnekonflikter.

Et eksempel på hva som ville skjedd uten navnerom: Standardbiblioteket inneholder beholderen `vector`. Dersom vi ønsket å lage vår egen matematiske vektor kunne vi ikke kalt den `vector` uten navnerom.

```
#include <vector> // Inkluderer beholderen vector fra standardbiblioteket
// Definerer vår egen matematiske vector
class vector { double x, y; };
int main() {
    vector v; // Hvilken vector er dette?
    return 0;
}
```

Et eksempel på slik det faktisk er: For å unngå slike navnekonflikter defineres alle navn i standardbiblioteket i navnerommet `std`.

```
#include <vector> // Inkluderer beholderen vector fra standardbiblioteket
class vector { double x, y; };
int main() {
    std::vector v; // Beholderen fra standardbiblioteket
    vector u; // Vår matematiske vector
    return 0;
}
```

Vi kan også definere vårt eget navnerom for å skille navnene fra hverandre:

```
#include <vector> // Inkluderer beholderen vector fra standardbiblioteket
namespace math { // Definerer vårt eget navnerom
class vector { double x, y; };
} // namespace math slutt
int main() {
    std::vector v; // Beholderen fra standardbiblioteket
    math::vector u; // Vår matematiske vector
    return 0;
}
```

Dersom det ikke er noen fare for en navnekonflikt kan vi hente navnene fra et navnerom inn i det globale navnerommet ved å bruke `using namespace`.

```
#include <iostream> // std::cout og std::endl
int main() {
    // Må skrive std:: for å fortelle hvilket navnerom cout er i
    std::cout << "Hello world!" << std::endl;
    return 0;
}

#include <iostream> // std::cout og std::endl
using namespace std; // Henter alle navnene i std inn i det globale navnerommet
int main() {
```

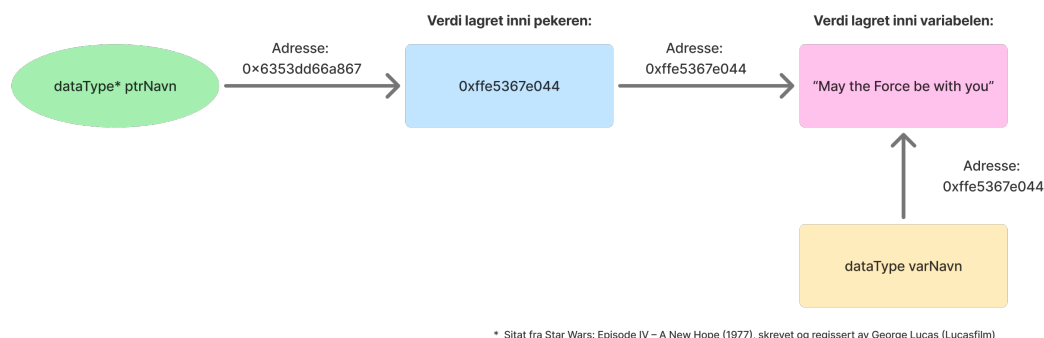
```
cout << "Hello world!" << endl; // Kan droppe std::
return 0;
}
```

Det er generelt frarådet å bruke `using namespace` i headerfiler. Dette er fordi når headerfilen inkluderes flere steder, vil alle navnene i std-biblioteket automatisk bli importert til det globale navnerommet disse stedene, uten at du nødvendigvis er oppmerksom på det. Dette kan føre til potensielle navnkonflikter.

I denne øvingen brukes det vanlige pekere. I C++ burde smartpekere, som `std::unique_ptr`, brukes mest mulig (og denne typen pekere blir dere kjent med i neste øving), men det er viktig å bli ordentlig godt kjent med vanlige pekere og hvordan de fungerer først. Du lurer kanskje på: *Hvorfor bry oss med pekere?! Det er jo så mye styr!!!* og den siste oppgaven er et godt eksempel på akkurat hvorfor.

1 Pekere og Referanser (10%)

I forelesningen har dere møtt forskjellen på pekere og referanser. Denne deloppgaven vil vise hvordan bruk av minnet kan gi forskjellig output, og har som mål å gi litt mer intuitiv forståelse om hvordan denne fungerer. For mer informasjon om forskjellen på pekere og referanser, se til kapittel 16 i boken. Denne deloppgaven vil foregå i filen `PointerRef.cpp` og `PointerRef.h`.



Figur 1: Visuell representasjon av pekere.

- Se på funksjonene `addPointer` og `addReference`.** Her ser dere at `addReference` er ferdig skrevet, men at `addPointer` er blank. Skriv kode slik at `addPointer` også kan legge til 20 til verdien `ptr` peker på.
- Lag en funksjon `bool valueComparison(int* ptr, int& ref)`.** Dere skal aksessere pekeren med den aktuelle verdien vi skal sammenligne med referansen. Denne funksjonen skal returnere en `true` dersom disse to `int` veridene er **like**. Funksjonen skal være medlem av `PointerRef` klassen, på samme måte som `addPointer` og `addReference`.
- Lag en funksjon `bool addressComparison(int* ptr, int& ref)`.** Ta inn en peker og en referanse og sammenlign verdien til pekeren, og adressen til referansen. Denne funksjonen skal returnere en `true` dersom disse to addressene er **like**. Denne skal også være medlem av `PointerRef` klassen.

Hint: For å kunne bruke `std::cout`, må man huske å ha `#include <iostream>`.

Tips: For å sjekke om koden i oppgavene over (og de dere lager i senere oppgaver) gjør som den skal er det alltid lurt å teste. Dette er enkelt å gjøre i `main`, eksempelvis ved å opprette en `int` (eller flere) for å så lage både en peker og en referanse til den. Så er det bare å teste funksjonene dere har laget:)

Nyttig å vite: pass-by-pointer

Når vi ønsker å bruke pass-by-pointer må vi spesifisere det i funksjonsdeklarasjonen.

```
//funksjon som tar inn en peker til en int
void func(int* pointer);
```

Dersom vi skal gjøre et kall på `func(int*)` må vi gi en peker som argument. Husk at en peker egentlig bare er en adresse til en variabel, adressen får vi ved å bruke adresseoperatoren, `&`.

```
int a{0};
func(&a); //&a = adressa til a
```

2 Atoi med Peker (15%)

Nyttig å vite: char* og C-streng

I C++ kan tekst lagres på flere måter. En måte er å bruke C-strings, som er den “gamle” C-måten å lagre tekst på.

En `char` er ett tegn, f.eks. `'A'`, `'x'` eller `','`. Vi kan definere en `char` slik:

```
char c = 'A';
```

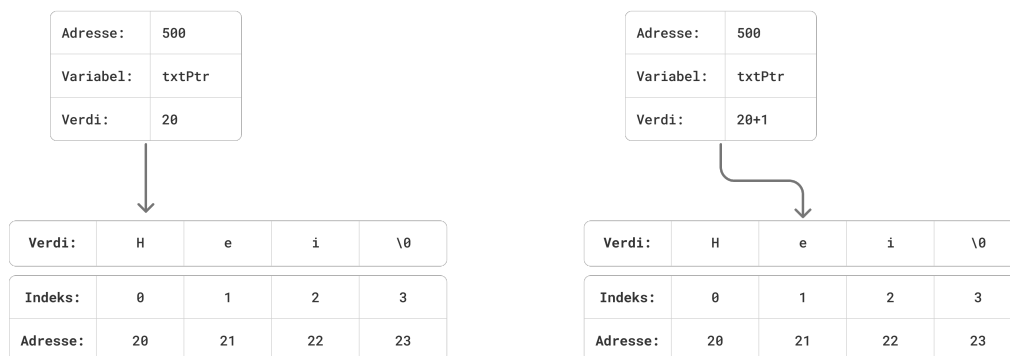
Tekst er bare en sammensetting av flere `char` og en C-string er et array av `char` som alltid slutter med tegnet `'\0'` (null-terminator). Eksempelvis:

```
const char* txtPtr = "hei"; // lagres som: 'h' 'e' 'i' '\0'
```

Dette betyr at du kan gå gjennom teksten tegn for tegn helt til du møter `'\0'`:

```
for (int i = 0; txtPtr[i] != '\0'; ++i) {
    // les txtPtr[i]
}
```

Under kan dere se en visuell representasjon av at vår `txtPtr` flyttes framover i minnet. Pegeren har selv en adresse (her lik 500, men dette er bare for å gi en ide av dette kan tenkes på som et tall), og verdien til pekeren er 20 som er adressen til den første bokstaven i vår C-streng. For å så hente ut neste bokstav, `e`, så legges én til verdien av pekeren (adressen til pekeren forblir den samme).



Når du ser en funksjon som returnerer `char*`, betyr det at den returnerer en **peker** som peker på et sted i minnet der det ligger et tegn (ofte starten på en C-streng "hei"). Hvis man bare skal lese fra minnet er det lurt å bruke `const char*` som sørger for at man ikke kan endre på strengen.

Du kan lese mer om dette i kapittel 16.3 i læreboken.

I denne oppgaven skal du jobbe med pekere og hvordan man kan "gå gjennom" en streng tegn for tegn i minnet. Målet er å implementere en forenklet variant av funksjoner som ofte kalles `atoi` (ASCII-to-integer): den leser et heltall som står først i en streng, og returnerer både tallet som ble lest og en peker til hvor lesingen stoppet. Denne oppgaven skal utføres i **Atoi.cpp**.

- a) Implementer funksjonen `const char* parseInt(const char* ptr, int* val)` i **Atoi.cpp**. Her skal outputet være en peker. Under er det lagt ved en pseudokode, som kan være til hjelp med å implementere funksjonen.

```
const char* parseInt(const char* ptr, int* outIntVal) {
    //int sign
    //int num

    //Hvis første tegn i ptr er '-', sett sign til -1, så gå til neste tegn

    //Ellers, sett sign til +1

    //Sett num til 0

    //While nåværende char er en int, konverter char til tall og legg til
    //tallet i slutten av "num". Gang tallet med 10, så gå til neste tegn

    //outIntVal peker nå på sign * num

    //Returner peker til første tegn etter tallet
}
```

Tips: Her kan det være nyttig å bruke de forhåndsdefinerte funksjonene `isDigit(char c)` og `characterToDigit(char c)`.

3 Kopiering av pekere (15%)

Merk at vi i denne oppgaven skal fremprovosere noen feil, så programmet ditt kan og skal oppføre seg litt rart. Koden i denne oppgaven skal skrives/redigeres på i filene `Dummy.h` og `Dummy.cpp`.

- a) Gitt følgende kode, hva forventer du at skrives ut når `dummyTest` kjører?
Denne oppgaven besvares i INGINious.

Dummy.h

```
#pragma once
#include <iostream>
#include <utility>

class Dummy {
public:
    int *num;
    Dummy() {
        num = new int{0};
    }

    ~Dummy() {
        delete num;
    }
};
```

Dummy.cpp

```
#include "dummy.h"

void dummyTest() {
    Dummy a;
    *a.num = 4;
    Dummy b{a};
    Dummy c;
    c = a;
    std::cout << "a: " << *a.num << '\n';
    std::cout << "b: " << *b.num << '\n';
    std::cout << "c: " << *c.num << '\n';

    *b.num = 5;
    *c.num = 1;

    std::cout << "a: " << *a.num << '\n';
    std::cout << "b: " << *b.num << '\n';
    std::cout << "c: " << *c.num << '\n';
}
```

- b) Fjern kommentarene (`/*` og `*/`) i toppen og bunnen av `dummy.h` og `dummy.cpp` slik at koden i filene ikke er kommentert ut. Kall deretter `dummyTest()` fra `main()`.

NB!: Ikke fjern kommentarene `// BEGIN: 3x` og `// END: 3x`, de trengs for å rette de andre deloppgavene i denne oppgaven.

Stemte output med det du forventet? Kan du forklare hvorfor/hvorfor ikke?

Denne oppgaven besvares i INGINious.

NB: Programmet krasjet sannsynligvis i slutten av `dummyTest`. Kan du forklare hvorfor?

- c) **Implementer kopikonstruktøren for Dummy-klassen.** Den skal ta inn en konstant referanse av Dummy-klassen med navn `rhs`. Dette skal gjøres inni klassesdeklarasjonen til klassen Dummy.

Hint: Det kan være lurt å se i vedlegget dersom du er usikker på hvordan du skal gjøre dette. Det står også mer om dette i kapittel 17.4 i boken.

Kjør programmet - skriver programmet ut noe annet enn forrige gang?

Programmet krasjer trolig fremdeles, da vi ikke har overlaster kopioperatoren (`operator=`) enda.

[`rhs` og `lhs`] Du lurer kanskje litt på hvorfor vi insisterer på å bruke variabelnavn satt sammen av tilfeldige bokstaver som `rhs` og `lhs`. Disse er faktisk forkortelser; `rhs` betyr *Right-hand side* og `lhs` betyr *Left-hand side*. Det er ganske vanlig å bruke dem når du overlaster operatører, f.eks. `lhs = rhs`.

- d) **Implementer `operator=` for Dummy.** Den skal ta inn et objekt av klassen Dummy med navn `rhs`. Du kan velge selv om du har lyst til å ta inn `rhs` som referanse eller bruke "copy-and-swap" teknikken. Dette skal også gjøres inni klassesdeklarasjonen til klassen Dummy.

Programmet skal nå kunne kjøre fra start til slutt uten å krasje. Skriver programmet nå ut det du forventet?

En gammel og god cipher som virkelig ikke har tålt tidens tann, er Cæsar cipheren. Denne går ut på at hver bokstav i alfabetet flyttes X plasser. Vi kaller dette å rotere koden. Så hvis A har plass 1, B plass 2 etc, vil man med en $ROT=3$ kryptere meldingen slik at A blir erstattet med D. Dette gjøres ved at A, med plass 1, flyttes til plass 4 (altså $1+3$), som er D. La oss ta ett eksempel. Si at du skal sende en melding der du avslører at favorittbandet ditt, med rette, er ABBA. Men dette er strengt klassifisert informasjon, så du velger en CC med $ROT = 2$. Da skal hver bokstav flyttes ned to hakk, så ABBA blir CDDC. Om vi nå ønsker å ta dette ett hakk videre, kan vi signere meldingen med den opprinnelige tallverdien til ABBA, $1+2+2+1=6$.

Cæsar cipheren er med andre ord ikke verdens vanskeligste å knekke, dersom man har litt tid og energi til å lete gjennom mulige konfigurasjoner. For denne oppgaven derimot så har vi gjemt den krypterte meldingen i flere lag med surr og tull. I csv filen **cipher.csv** (ikke fortvil om du ikke finner filen - den skal vi lage i oppgave a), har vi nesten 630 000 linjer med bokstaver og tall, som vi har gjemt en kode i, så her anbefaler vi å ikke teste grensene på hvor mye ntnu printerne tåler på en dag, dersom det skulle falle noen inn å prøve å løse dette manuelt. Denne oppgaven skal vi nå bruke pekere for å manipulere csv filen, og finne fram til riktig melding ved pekere, filmanipulasjon og en god dose forkjærlighet for logikk og vimseri.

Alle deloppgavene du skal implementere, skal gjøres i **CChiper.cpp**. Det kan være lurt å se gjennom medlemsvariablene til **DeCipher** for å få en ide om hvilke variabler du kan (og burde) bruke for å løse oppgavene.

- a) **Gå til filen CSVGenerator.cpp.** Vår hemmelige agent Rosenkål, har i feltet lurt seg unna med en viktig kodesnutt. Denne koden, samlet i **CSVGenerator.cpp**, kan generere meldingen som vi ønsker å knekke. Til vår felles fortvilelse så fant agenten vår fort ut at det er umulig å knekke den hemmelige meldingen direkte - så vi, hovedkontoret for hemmelige meldinger og amatørrell spionasje, starter med å generere CSV filen som inneholder meldingen.

I denne deloppgaven skal dere dermed kompilere og kjøre **CSVGenerator.cpp**. Resultatet skal være at dere har skapt en fil kallt **cipher.csv**, som skal være rundt 1 GB stor.

Nyttig å vite: compile-w-terminal

Om du er en av oss som syns terminal er gøy, kan du kompilere og kjøre CSV-generatoren herfra.

Start med å gå inn i csv-mappa med `cd` (change directory). Dette er en vanlig terminal-kommando (for mer intro til Git, se vedlegget). Deretter bruker vi `g++`, som er GNU sin C++-kompilator, til å kompilere **CSVGenerator.cpp** og lage et kjørbart program:

```
cd csv

g++ -std=c++17 CSVGenerator.cpp -o CSVGenerator

./CSVGenerator

cd ..
```

Flagget `-std=c++17` sier hvilken C++-standard som brukes, mens `-o CSVGenerator` bestemmer navnet på programmet som blir laget. Så kan du kjøre programmet med `./CSVGenerator`. Husk å til slutt gjøre `cd ..` for å komme tilbake til prosjektmappen.

- b) **Gå til filen CCipher.cpp.** Nå har vi kommet til et spektakulært gjennombrudd i de hemmelige tjenestene - cipher.csv filen er tilgjengelig og klar til knekking! Dessverre er

denne på størrelse av ca 1 GB, og inneholder 8x630 000 ord, og vi vet at ikke alle ordene er den hemmelige meldingen. Ordene er splitta av delimatoren ",". Nå må vi finne på noe lurt for å finne ut av hvilke ord som er den krypterte meldingen.

Begynn med å implementer funksjonen `jumpToStartOfNextLine()`. Funksjonen skal endre medlemsvariabelen `nextCharacterIndexToProcess` slik at den peker på første tegn etter linjeskiftet, altså ved å finne `'\n'` på den nåværende linjen (hver linje avsluttes med en `'\n'`). Dette vil bli nyttig når vi skal gå gjennom teksten linje for linje!

- c) **Videre har vi fått et hint av vår skarpsinnede agent Nepeskrell, om at de hemmelige meldingene kun finnes på linjer som er oddetall.** Dvs linje 1, 3, 5, etc. **Implementer funksjonen `jumpToStartOfNextOddLine()`.** Funksjonen skal endre `nextCharacterIndexToProcess` slik at den går fra nåværende oddetallslinje til neste oddetallslinje (altså hopp over én linje). Her kan det være lurt å bruke funksjonen du implemenerte i b) og husk å pass på om du er på slutten av filen!
- d) **Agentene Rosenkål og Nepeskrell har lagt hodene i bløt, og har sammen gjort et hårreisende funn.** På hver relevante linje står det potensielle meldingsordet alltid i siste kolonne! **Implementer funksjonen `getLastWordOnCurrentLine(char*& lastWord, int& lengthOfWord)`.** Funksjonen skal endre på pekeren `lastWord` slik at den peker på første bokstav i det siste ordet i linja `inni csvFileContents`. I tillegg skal `lengthOfWord` settes lik lengden (antall bokstaver) av dette ordet. (Husk å ekskluder `'` og `'\n'`). **Viktig:** Pass på at `nextCharacterIndexToProcess` ikke modifiseres av denne funksjonen.
- e) **Nå har vi fått en lang melding med potensielle ord - på tide å sortere bort søppelordene!** Implementer funksjonen `void decodeCipher(int ROTKey)`. Funksjonen tar inn en rotasjon (ROT) og skal bruke denne til å dekode slik beskrevet i introduksjonen av oppgaven. `decodeCipher` skal altså gå gjennom hele `csvFileContents` og finne siste ord på hver oddetallslinje, sjekke om lengden av ordet er et partall, dekode hver karakter med `ROTKey` og legge den inn i `decodedMessage` (som vi allerede har allokert minne til i konstruktøren) OBS: Meldingen er på engelsk, og det er dermed et alfabet med lengde 26, ikke 29. **Tips:** Husk å begynne med å nullstille `nextDecodedCharacterIndex` og `nextCharacterIndexToProcess` for at funksjonen skal kunne kalles flere ganger.
- f) **Spionkontorets leder, Major Kloss, har fått blod på tann, og vil løse den krypterte meldingen.** I stedet for å sjekke hvert eneste ord manuelt, så har du fått vite at meldingen er skrevet med CC, og $ROT = 6 < \text{et primtall} < 20$. (altså 7, 11, 13, 17 og 19). Dekod meldingen for de relevante rotasjonene i `main` ved å bruke funksjonen du lagde i e), `decodeCipher(int ROTKey)`, og **fill inn riktig rotasjon i multiple choice seksjonen av INGINious**. Du kan skrive meldingen til terminal ved å bruke medlemsfunksjonen `getDecodedMessage()`.
- g) I `DeCipher` allokerer konstruktøren minne med `new[]`. Forklar hva som skjer hvis vi ikke skriver en destruktør som frigjør dette minnet og forklar kort hvorfor vi må bruke `delete[]` og ikke `delete`. **Denne oppgaven skal besvares i INGINious.**

I `DeCipher` finner du også funksjonen `benchmarkSpeed()`. Den sammenligner hvor lang tid det tar å dekode teksten ved bruk av pekere og ved bruk av `std::getline` og `std::stringstream`. Hvis du kjører `benchmarkSpeed()`, vil du se at peker-varianten blir **mye raskere**, spesielt når filen er stor. Siden pekere ofte krever litt mer arbeid og nøyaktighet er det ikke alltid slik at du vil velge pekere fremfor `std::string` og strømmer. Men til gjengjeld kan du få svært mye igjen i ytelse når du bruker dem riktig. Dette er en av grunnene til at vi fortsatt bryr oss om pekere i C++.

Vedlegg: Forklaring av dynamisk minne, kopikonstruktør, operator = og intro til Git

Dynamisk minnehåndtering

Hittil i øvingsopplegget har vi blant annet brukt `std::vector`, som tar seg av minneallokering automatisk. Likevel er det ikke alltid `std::vector` egner seg eller er tilgjengelig, f.eks. i innebygde systemer (embedded systems). Dermed er det nyttig å kjenne til bruk av dynamisk allokert minne, uten `std::vector`.

Du lurer nå kanskje på hva som er poenget med å lære seg dynamisk minnehåndtering når vi kan bruke `std::vector` istedenfor. Vi lærer det bort fordi det gir en forståelse for hvordan f.eks. `std::vector` er bygd opp. `std::vector` kan fra nå av tenkes på som noe annet enn en «black-box», du lærer hva som foregår «bak kulissene». Du lærer også om lavnivå programmering, som kan være nyttig å kunne hvis du f.eks. skal jobbe med innebygde systemer.

Vi kan reservere minne mens programmet kjører, men vi kan ikke lage nye variabelnavn «på sparket». Derfor er pekere vesentlig når vi skal bruke dynamisk minne. For å reservere dynamisk minne i C++ bruker vi operatoren `new`, som returnerer en peker til begynnelsen av det minnet programmet har fått tildelt. Uten å ta vare på denne pekeren i en pekervariabel har vi ingen måte å lese fra eller skrive til dette minnet.

Merk at når vi reserverer minne med `new` kan ikke kompilatoren vite hvor lenge programmet har bruk for minnet, og vi må selv ta ansvar for å rydde opp etter oss. Det gjør vi med `delete`-operatoren:

```
void newAndDelete() {
    // Allokert minne for en int med new
    int* x = new int {};

    std::cout << "Skriv inn et heltall: ";
    std::cin >> *x;
    std::cout << "Takk! Du skrev: " << *x;

    // Frigjør minnet når vi er ferdig med det
    delete x;
    x = nullptr;
}
```

Dersom man ikke gjør dette vil det over programmets levetid bli allokert mer og mer minne som ikke blir frigitt, helt til datamaskinen er tom for minne og programmet krasjer. Dette kalles en minnelekkasje. Minnelekkasjer er også i mange tilfeller en sikkerhetsrisiko. Det oppstår lett minnelekkasjer når man bruker dynamisk minne, så beste praksis er å kun bruke det der det er absolutt nødvendig. I eksempelet over er bruk av dynamisk minne helt unødvendig, men det er kun ment som et trivielt eksempel for å demonstrere syntaksen.

Konvensjonen er at den delen av programmet som reserverer minnet «eier» minnet og er derfor også ansvarlig for å frigjøre minnet. I prosedural kode vil det som regel bety at funksjonen der minnet blir reservert også frigjør minnet, og i objektorientert kode (som bruker klasser) betyr det at man allokterer minne i konstruktøren til et objekt og frigjør det i destruktøren.

For å allokere minne til en matrise gjør man følgende:

```

void dynamic2DArrayExample() {
    int n = 0;
    std::cout << "What dimension should the matrix have? ";
    std::cin >> n;

    // Her skal vi allokere minne for en array med typen double og størrelse n*n

    // Allokterer minne for en array med plass til n andre arrays (pekere)
    double **numbers = new double*[n] {};

    for(int i = 0; i < n; i++) {
        // Allokterer minne for n arrays med typen double og størrelse n
        numbers[i] = new double[n] {};

        for(int j = 0; j < n; j++) {
            std::cout << "Input number: ";
            std::cin >> numbers[i][j];
        }
    }

    std::cout << "You entered: " << '\n';
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            std::cout << numbers[i][j] << " ";
        }
        std::cout << '\n';
    }
    std::cout << '\n';

    // Frigjør minne når vi er ferdige med det
    for (int i = 0; i < n; i++){
        // Frigjør minne til pekerene inni numbers
        delete[] numbers[i];
        numbers[i] = nullptr;
    }
    // Frigjør minne til numbers
    delete[] numbers;
    numbers = nullptr;
}

```

Legg merke til at når vi frigjør minnet som er reservert for et array så må vi bruke `delete[]` og ikke `delete`!

Introduksjon til klasser med dynamisk allokerede medlemsvariabler

Det oppstår et spesialtilfelle vi må ta hensyn til når vi har dynamisk allokerede medlemsvariabler i en klasse.

Se på følgende eksempel:

```

class Example {
private:

```

```

    int *anInt;

public:
    Example(int i) {
        anInt = new int{i};
    }

    Example() {
        anInt = new int{0};
    }

    ~Example() {
        delete anInt;
    }

    int get() const { return *anInt; }
};

```

Ved første øyekast ser dette ut som en veldefinert klasse. Den konstrueres riktig, og dersom konstruktøren fullfører kan vi være sikre på at vi alltid har et initialisert heltall lagret i `anInt`. Problemet vi ser etter dukker først opp når vi skriver denne koden:

```

Example a{ 5 };
Example b;

b = a; // Hva skjer her?

```

Når vi skriver `b = a` kaller vi på operator `=`. Vi har imidlertid ikke definert noen slik operator. Operatoren som blir kalt er da en som kompilatoren lager automatisk. Denne operatoren tar en binær kopi, som vil si at den kopierer medlemsvariabel for medlemsvariabel uten å bry seg om noen av dem er dynamisk allokert.

Dette er et problem fordi den eneste medlemsvariabelen i klassen `Example`, `anInt`, er en peker. Dermed er det *pekeren* og *ikke* minnet pekeren peker til som blir kopiert. Resultatet er at `b` og `a` begge har en peker som peker til nøyaktig samme instans av det dynamisk allokerede heltallet!

La oss ta en kikk på følgende kodesnutt:

```

Example a{ 5 };
if (a.get() > 0) {
    Example b = a;
    std::cout << b.get();
}

```

Hva skjer i koden over?

Først legger vi merke til at blokken til if-setningen kommer til å bli kjørt siden vi vet at `a` er 5, og derfor større enn null. Vi kopierer `a` til `b`, `b` skrives ut, og idet vi forlater blokken vil `b` bli destruert. Med andre ord vil `b` sin destruktør bli kalt. Husk fra definisjonen av klassen at destruktøren sletter minnet som er dynamisk allokert og pekt på av `anInt`.

Hva skjedde med `a` sin `anInt`? Siden `b` og `a` hadde pekere til det samme dynamisk allokerede heltallet, vil pekeren i `a` nå være ugyldig.

Hvordan kan vi fikse dette? Det vi har beskrevet over kalles en *grunn kopi* (shallow copy), og er ikke alltid en dårlig løsning. I så fall må man holde nøye orden på hvor mange pekere som peker til det samme minnet.

Løsningen er å implementere såkalt *dyp kopiering* (deep copying) i `operator =`. En dyp kopi av en peker kopierer også *minneområdet* som pekeren peker til. Når vi kopierer et objekt vil medlemsvariablene i det nye objektet peke til nye, dynamisk allokerede minneområder, og vi unngår problemene beskrevet over.

Kopikonstruktør og `operator =`

For å implementere dyp kopiering for `Example`-klassen, må vi legge til eksplisitte implementasjoner av en *kopikonstruktør* og `operator =`.

Kopikonstruktøren har alltid formen `Klassenavn(const Klassenavn& other)`. Det er denne som kalles når du initialiserer et objekt med et annet objekt av samme type, for eksempel ved å skrive `Example b{a};`.

```
Example(const Example& other) {
    this->anInt = new int{}; // Vi allokere det vi trenger av minne
    *anInt = other.get();    // Vi kopierer verdier fra other til this
}
```

Tilordningsoperatoren kan implementeres på forskjellige måter. Vi kan ta inn høyresiden som en `const` referanse. Da frigjør vi først det gamle minnet til lhs, deretter allokere nytt minne og kopiere over verdiene fra høyresiden.

```
Example& operator=(const Example& other) {
    delete anInt;
    anInt = new anInt = new int{};
    *anInt = other.get();
    return *this;
}
```

En annen vanlig og god teknikk å bruke er «copy-and-swap». Kort fortalt bruker vi en call-by-value parameter for høyreoperanden, da tar vi inn en kopi (`rhs`) som automatisk opprettes med bruk av kopi-konstruktøren.

Deretter swapper vi alle medlemsvariabler (inkl. pekere til allokert data) mellom de to objektene. Dermed får `this` (venstresiden) tilordnet høyresidens verdier (og allokerede data) og `rhs` ender opp med det som skal slettes. Siden `rhs` er en lokal variabel vil destruktøren dens automatisk bli kalt når den går ut av scope.

```
Example& operator=(Example rhs) {
    // Vi tar inn rhs som call-by-value
    std::swap(anInt, rhs.anInt) // Vi swapper pekere
    return *this;
}
```

`std::swap` ligger i biblioteket `<utility>`, og er veldig nyttig i dette tilfellet, den er deklart slik:

```
template<class T> void swap( T& a, T& b);
```

Deklarasjonen viser at `swap` tar inn to referanser som argumenter, disse kan være av hvilken som helst type, inkludert brukerdefinerte typer, men `a` og `b` må ha samme type. `std::swap` returnerer ingenting, men den bytter om `a` og `b`, det vil si at alle medlemsvariabler flyttes fra den ene til den andre. Dette gjelder også dynamisk allokert minne, da vil de bytte pekere til de ulike minneområdene. Funksjonen har samme effekt på `a` og `b` så det har ingenting å si hvilken variabel som kalles først i argumentlisten. Eksempelet under viser hva som skjer når man kaller `std::swap` på to objekter fra klassen vår `Example`.

```
Example a{1};
Example b{2};
std::cout << "Initial values: a= " << a.get() << "b= " << b.get() << '\n';
std::swap(a, b);
std::cout << "Values after swap: a= " << a.get() << "b= " << b.get() << '\n';
}
```

Dette vil bli skrevet ut når koden kjøres:

```
Initial values: a= 1 b= 2
Values after swap: a= 2 b= 1
```

Vi ser at verdiene til `a` og `b` ble byttet om som vi ville og de peker nå til motsatt minneområde av det de gjorde i sted. Husk at i overlastingen av `operator=` er `rhs` en call-by-value parameter og dermed en kopi av det objektet vi setter `this` lik. Dermed vil `swap` bytte minneområdet til kopien og ikke det faktiske objektet og det er sånn vi unngår flere pekere til samme minneområde.

Introduksjon til Git

Grunnleggende forståelse av Git er praktisk dersom en bedriver programmering, da disse tillater raske kommandoer og god oversikt over både filer, handleringer og historikk. Senere i livet generelt kan det være at dere samarbeider på programmeringsprosjekt, og da er en typisk plattform for å dele kode github. Så denne delen av vedlegget sikter mot å gi en introduksjon og grunnforståelse av Git, for de som er interesserte.

Så, hvordan bruke git? Git brukes og aksesseres via terminal, som finnes på alle datamaskiner. Windows brukere benytter ofte powershell, men det kommer ann på bruk og smak og behag (Windows har heller ikke alltid installert Git fra start). Så hva er terminal? Rett og slett datamaskinen din i ett noe råere format - du kan bruke terminal til alt det datamaskinen din kan gjøre, men brukergrensesnittet gitt av programmer er ofte lettere å lese for mennesker. Si f.eks. at du vil skrive et dokument, så kunne du ha brukt et tekstredigeringsprogram som Microsoft Word, men du kunne også opprettet et dokument og skrevet i det direkte fra terminal. Så terminal kan gjøre alt det datamaskinen din og programmer kan gjøre - bare med et noe mindre brukervennlig grensesnitt.

Hvorfor bruker vi da tid på terminal og git om vi har programmer vi kan bruke i stedet lur du kanskje på. Ganske enkelt fordi at når man programmerer, så vil det ofte være lettere å benytte seg av terminal, fordi at man ofte kan beholde oversikten over de ulike filene og kommandoene du ønsker å kjøre i filene lettere.

Så dersom du vil hoppe litt mer inn på git, så er det en rekke med nifty kommandoer du skal på se på.

- `cd mappenavn`
Change directory. Denne bruker du når du vil gå inn i en mappe.
- `cd ..`
Denne tar et steg ut fra mappen, til overstående mappe.
- `ls`
Denne lister opp mappene og filene på det nivået du er på.
- `git init`
Initialiserer et nytt Git-repository i den gjeldende mappen.
- `git clone <url>`
Kloner et eksisterende repository fra en URL til lokal.
- `git status`
Viser endringer i prosjektet, for eksempel hvilke filer som er endret og klare til å committes.
- `git add <filnavn>`
Legger til filer i staging-området slik at de inkluderes i neste commit.
- `git commit -m "<commit melding>"`
Lager et nytt commit med en beskrivende melding (god kodeskikk er å alltid ha korte, beskrivende meldinger så du raskt kan se hva som har blitt lagt til).
- `git push`
Sender lokale commits til en ekstern server, oftest GitHub.
- `git pull`
Henter og slår sammen endringer fra et eksternt repository. For å unngå merge konflikter er det alltid lurt å pulle endringer først, slik at du unngår store konflikter.
- `git log`
Viser commit historikken.
- `git branch`
x Viser eksisterende grener. Bruk `git branch <navn>` for å lage en ny branch.
- `git checkout <branch navn>`
Bytter til en annen branch.
- `git merge <branch navn>`
Slår sammen en annen branch inn i den aktive grenen.

Over, så står f.eks. `git add <filnavn>` med "<>" rundt filnavn. Alle steder det brukes "<>", er disse ment å erstattes. Så dersom eksempelet her hadde vært ekte, og jeg ville adde en fil som heter `filenMin.py`, ville jeg ha skrevet `git add filenMin.py` i terminal.